

2. Bölüm - Java Script Kod Yapısı

Java Script'e console çıktısı verme ve Alert Penceresi açma

```
console.log("merhaba Dünya");  
console.log(2+3);
```

* Java scripte console.log
konsola yazılıyor.

```
alert("Sayfa açıldı");
```

* Alert penceresi açmak isteniyorsa
"alert" kullanılır.

Değişkenler

* Java scripte 2 farklı değişken vardır.

- let
- const

1. let

- let ile tanımlanan değişken değeri daha sonra değişebilir.

```
let isim = "Ahmet";  
console.log(isim);
```

→ Ahmet

```
let puan = 50;  
console.log(puan); → 50  
let puan = 75;  
console.log(puan); → 75
```

2. const

- const ile tanımlanan değer daha sonra değiştirilemez.

```
const okul = "İnönü Üniversitesi";  
console.log(okul);
```

Veri Tipleri

* Java Script'te başlıca veri tipleri:

Veri Tipi	Açıklama
String	Metin
Number	Sayı
Boolean	true / false
Undefined	Değer atanmamış
Null	Bos değer

* Java scriptte otomatik
tip dönüşümü bulunmaktadır.
* Javascriptte bir değişkenin
tipini öğrenmek için "typeof"
kullanılır.

```
1 let ad = "Zeynep";  
2 let yas = 21;  
3 let ogrencimi = true;  
4 let veri = null;  
5 let durum;  
6  
7 console.log(typeof ad); → String  
8 console.log(typeof yas); → Number  
9 console.log(typeof ogrencimi); → Boolean  
10 console.log(typeof durum); → Undefined  
11 console.log(typeof veri); → Null
```

Operatörler

* Operatörler veri üzerinde işlem yapmamızı sağlar.

a. Aritmetik Operatörler

```
1 let a = 10;  
2 let b = 3;  
3  
4 console.log(a + b); → 13  
5 console.log(a - b); → 7  
6 console.log(a * b); → 30  
7 console.log(a / b); → 3.23  
8 console.log(a % b); → 1
```

b. Karşılaştırma Operatörleri

```
1 let x = 5;  
2 let y = 8;  
3  
4 console.log(x > y);  
5 console.log(x < y);  
6 console.log(x == y);  
7 console.log(x === y);
```

Listing 8: Karşılaştırma operatörleri

== → sadece değer karşılaştırır
=== → veri ve değer

c. Mantıksal Operatörler

```
1 console.log(true && false);  
2 console.log(true || false);  
3 console.log(!true);
```

Listing 9: Mantıksal operatörler

Kullanıcıdan veri alma

* JavaScriptte kullanıcıdan veri almak isteniyorsa prompt() kullanılır.

```
let isim = prompt("Adınızı Giriniz:");  
console.log(isim)
```

```
let sayi1 = Number prompt("Bir sayı giriniz:");  
let sayi2 = Number prompt("Bir sayı giriniz:");  
console.log("Toplam", sayi1 + sayi2);
```

* prompt() ile kullanıcıdan veri alırken tip dönüşümü gerekir. prompt verdiği metin olarak alır.

Koşul Yapısı

1. if ve else

```
let yas = 18;
if (yas >= 18) {
  console.log("Ehliyet alabilir");
} else {
  console.log("Ehliyet alamaz.");
}
```

2. else if

```
let not = 15;
if (not >= 50) {
  console.log("Geçti");
} else if (not < 50) {
  console.log("Kaldı");
} else {
  console.log("Gecersiz");
}
```

3. switchcase

```
let gun = 3;
switch (gun) {
  case 1:
    console.log("Pazartesi");
    break;
  case 2:
    console.log("Salı");
    break;
  case 3:
    console.log("Çarşamba");
    break;
  default:
    console.log("Gecersiz gün");
}
```

" "

Döngüler

1. for Döngüsü

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

2. while Döngüsü

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

3. Do while Döngüsü

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i < 5);
```

Fonksiyonlar

1. Parametrelili Fonksiyon

```
function toplama(a,b){  
  console.log(a+b);  
}  
toplama(3,5);
```

2. Parametresiz Fonksiyon

```
function selamVer(){  
  console.log("merhaba");  
}  
selamVer();
```

3. Değer Döndüren fonksiyon

```
function kareAl(a){  
  return a*a;  
}  
let sonuc = kareAl(4);  
console.log("Sonuc = ",sonuc);
```

4. Arrow function

```
const kare = x => x*x;  
console.log(kare(5));
```

- Modern JavaScript fonksiyonları kısa şekilde kullanmak için kullanılır.

Diziler

```
let meyveler = ["Koyun", "Erik", "Muz"];  
console.log(meyveler);  
console.log(meyveler[0]);  
console.log(meyveler.length);  
meyveler.push("Kiraz");  
meyveler.pop();  
meyveler.shift();  
meyveler.unshift("Çilek");
```

- **push()**: Sona eleman ekler
- **pop()**: Sondan eleman çıkarır
- **shift()**: ilk " "
- **unshift()**: başa " " ekler.

Dizi elemanları döngüler yardımıyla tek tek işlenebilir.

```
1 let sayilar = [10, 20, 30, 40];  
2  
3 for (let i = 0; i < sayilar.length; i++) {  
4   console.log(sayilar[i]);  
5 }
```

Listing 25: for ile dizi elemanlarını yazdırma

```
1 let sayilar = [10, 20, 30, 40];  
2  
3 for (let sayi of sayilar) {  
4   console.log(sayi);  
5 }
```

Listing 26: for...of kullanımı

Nesneler

```
let ogrenci = {  
  ad: "Eren";  
  yas: 21;  
  bolum: "Bilgisayar Muhendisligi";  
}  
console.log(ogrenci.ad);  
console.log(ogrenci.yas);  
ogrenci.not = 30 → Özelik ekleme
```

Metin işlemleri

```
let metin = "JavaScript";  
console.log(metin.length);  
console.log(metin.toUpperCase());  
console.log(metin.toLowerCase());  
console.log(metin.includes("Script"));
```

*toUpperCase(): Büyük harfe çevirme
*toLowerCase(): Küçük " " " "
*includes(): Belirli bir harfi içeriyor mu?

Matematiksel işlemler

```
1 console.log(Math.round(4.7));  
2 console.log(Math.floor(4.7));  
3 console.log(Math.ceil(4.2));  
4 console.log(Math.random());
```

*round(): en yakın tam sayıya yuvarlar
*floor(): aşağı yuvarlama
*ceil(): yukarı " "
*Math.Random(): Random değer üretme

```
1 let rastgele = Math.floor(Math.random() * 10) + 1;  
2 console.log(rastgele);
```

16. DOM'a Giriş

DOM, web sayfasındaki HTML elemanlarının programlama yoluyla temsil edilmesini sağlar. JavaScript, DOM üzerinden sayfadaki elemanlara erişebilir ve bu elemanların içeriğini değiştirebilir.

```
1 <h1 id="baslik">Merhaba</h1>
```

Listing 32: HTML örneği

```
1 const baslik = document.getElementById("baslik");  
2 baslik.textContent = "JavaScript ile degisti";
```

Listing 33: DOM ile içerik değiştirme

17. Olaylar

Kullanıcının bir butona tıklaması, klavyeden yazı yazması veya fareyi hareket ettirmesi gibi işlemler olay olarak adlandırılır.

```
1 <button id="btn">Tikla</button>
2 <p id="mesaj">Bekleniyor...</p>
```

Listing 34: Buton ve mesaj alanı

```
1 const btn = document.getElementById("btn");
2 const mesaj = document.getElementById("mesaj");
3
4 btn.addEventListener("click", function () {
5     mesaj.textContent = "Butona basıldı";
6 });
```

Listing 35: click olayı

18. Form İşlemleri

Web sayfalarında kullanıcıdan veri almak için form elemanları kullanılır.

```
1 <input type="text" id="ad" placeholder="Adinizi yazın">
2 <button id="gonder">Gonder</button>
3 <p id="sonuc"></p>
```

Listing 36: Basit form örneği

```
1 const gonder = document.getElementById("gonder");
2 const ad = document.getElementById("ad");
3 const sonuc = document.getElementById("sonuc");
4
5 gonder.addEventListener("click", function () {
6     sonuc.textContent = "Merhaba " + ad.value;
7 });
```

Listing 37: Formdan veri alma

19. Fonksiyonlarla DOM Kullanımı

Fonksiyonlar ile sayfa etkileşimi birlikte kullanılabilir.

```
1 function selamla(isim) {
2     return "Merhaba " + isim;
3 }
4
5 const gonder = document.getElementById("gonder");
6 const ad = document.getElementById("ad");
7 const sonuc = document.getElementById("sonuc");
8
9 gonder.addEventListener("click", function () {
10     sonuc.textContent = selamla(ad.value);
11 });
```

Listing 38: Fonksiyon ve DOM kullanımı

20. Zamanlayıcılar

Belirli bir süre sonra veya belirli aralıklarla çalışın işlemler için zamanlayıcılar kullanılır.

20.1. setTimeout

```
1 setTimeout(function () {
2     console.log("3 saniye sonra calisti");
3 }, 3000);
```

Listing 39: setTimeout örneği

20.2. setInterval

```
1 let sayac = 0;
2
3 setInterval(function () {
4     sayac++;
5     console.log(sayac);
6 }, 1000);
```

Listing 40: setInterval örneği

21. Hata Ayıklama

Kod çalışırken oluşan hataları anlamak için konsol çıktıları ve hata mesajları kullanılır.

```
1 let ad = "Ahmet";
2 console.log(isimm);
```

Listing 41: Tanımlanmamış değişken hatası

Bu örnekte `isimm` adlı değişken tanımlanmamıştır. JavaScript hata verir ve hatanın bulunduğu satır bilgisi konsolda görüntülenir.

22. Asenkron Yapıya Giriş

Bazı işlemler hemen tamamlanmaz. Özellikle internetten veri alma işlemleri belirli bir süre gerektirir. Bu tür işlemler asenkron yapı ile ilgilidir.

```
1 fetch("https://jsonplaceholder.typicode.com/users/1")
2   .then(response => response.json())
3   .then(data => console.log(data));
```

Listing 42: fetch ile veri alma örneği

Bu örnekte internet üzerinden veri alınmakta, gelen veri JSON biçiminden dönüştürülmekte ve sonuç ekrana yazdırılmaktadır.

İleri Düzey Dizi İşlemleri

1. `for Each()`: Bir dizinin her elemanı için verilen işlemi uygular.

```
let sayilar = [10,20,30,40];
sayilar.forEach(function(sayi){
    console.log(sayi);
})
```

2. `map()`: Bir dizinin her elemanı için verilen işlemi yapar ve bir dizi olarak döndürür.

```
let sayilar = [10,20,30,40];
let kareler = sayilar.map(function(sayi){
    return sayi * sayi;
})
console.log(kareler)
```

3. filter(): Belirli bir koşulu sağlayan elementleri seçer

```
let sayilar = [1, 2, 3, 4, 5, 6];
let ciftSayilar = sayilar.filter(function(sayi) {
  return sayi % 2 === 0;
});

console.log(ciftSayilar);
```

4. find(): Koşulu sağlayan ilk elementi döndürür.

```
let ogrenciler = [
  { ad: "Ayse", not: 70 },
  { ad: "Mehmet", not: 85 },
  { ad: "Zeynep", not: 90 }
];

let sonuc = ogrenciler.find(function(ogrenci) {
  return ogrenci.not > 80;
});

console.log(sonuc);
```

5. some() ve every():

— some(): en az bir elementin koşulu sağlıyor sağlamadığını kontrol eder.

— every(): tüm elementlerin " " " " " "

```
let sayilar = [2, 4, 6, 8];

console.log(sayilar.some(function(sayi) {
  return sayi > 5;
}));
```

```
console.log(sayilar.every(function(sayi) {
  return sayi % 2 === 0;
}));
```

6. reduce(): Dizideki elementleri tek bir sonuca indirger.

```
1 let sayilar = [1, 2, 3, 4, 5];
2
3 let toplam = sayilar.reduce(function(toplam, sayi) {
4   return toplam + sayi;
5 }, 0);
6
7 console.log(toplam);
```

Metin ve Dizi Metodları

Metin Metodları

```
let metin = "  JavaScript Dersleri  ";

console.log(metin.trim());
console.log(metin.toUpperCase());
console.log(metin.toLowerCase());
console.log(metin.includes("Script"));
console.log(metin.replace("Dersleri", "Notlari"));
```

*trim(): Başında ve sonundaki boşlukları siler.

*replace(): İfadeyi içindeki stringi değiştirir.

Parçalama ve Birleştirme

```
1 let cumle = "JavaScript temel konulari";
2 let kelimeler = cumle.split(" ");
3
4 console.log(kelimeler);
5
6 let yeniMetin = kelimeler.join("-");
7 console.log(yeniMetin);
```

***split():** Bir metnin belirli bir karaktere göre parçalayıp diziye ekler.

***join():** Bir diziye tekfor nesneye dönüştürür.

Kesme işlemi

```
1 let kelime = "Bilgisayar";
2
3 console.log(kelime.slice(0, 5));
4 console.log(kelime.substring(5, 10));
```

***slice():** Metinden belirli bir kısmı keser.

***substring():** " " " " "

Nesnelerde ileri işlemler

1. İnce nesneler

```
1 let ogrenci = {
2   ad: "Ayse",
3   bolum: "Bilgisayar Muhendisligi",
4   adres: {
5     sehir: "Malatya",
6     ulke: "Turkiye"
7   }
8 };
9
10 console.log(ogrenci.adres.sehir);
```

2. Nesne dizisi

```
1 let ogrenciler = [
2   { ad: "Ayse", not: 70 },
3   { ad: "Mehmet", not: 85 },
4   { ad: "Zeynep", not: 90 }
5 ];
6
7 console.log(ogrenciler[1].ad);
```

3. Object.keys(), Object.values(), Object.entries();

```
1 let kitap = {
2   ad: "JavaScript",
3   sayfa: 350,
4   yazar: "Ali Yilmaz"
5 };
6
7 console.log(Object.keys(kitap));
8 console.log(Object.values(kitap));
9 console.log(Object.entries(kitap));
```

***Object.keys():** Nesnenin tüm anahtarlarını bir dizi olarak döndürür.

***Object.values():** Nesnenin tüm değerlerini döndürür.

***Object.entries():** Nesnenin Key-value ilişkisi kurur.

Destructuring

* Destructuring, dizi ve nesnelerdeki verileri kısa ve düzenli biçimde değişkenlere ayırma sağlar.

```
1 let renkler = ["kirmizi", "mavi", "yesil"];
2
3 let [birinci, ikinci, ucuncu] = renkler;
4
5 console.log(birinci);
6 console.log(ikinci);
7 console.log(ucuncu);
```

Listing 13: Dizi destructuring

```
1 let kullanıcı = {
2   ad: "Ahmet",
3   yas: 22,
4   sehir: "Malatya"
5 };
6
7 let { ad, yas, sehir } = kullanıcı;
8
9 console.log(ad);
10 console.log(yas);
11 console.log(sehir);
```

Listing 14: Nesne destructuring

Spread ve Rest Operatörü

1. Spread Operatör (...)

* Bir dizi veya nesnenin elemanlarını yaymak (cozmek) için kullanılır.

```
1 let sayilar1 = [1, 2, 3];
2 let sayilar2 = [...sayilar1, 4, 5];
3
4 console.log(sayilar2);
```

Listing 15: Spread operatörü

```
1 let kisi = { ad: "Ayse", yas: 21 };
2 let yeniKisi = { ...kisi, bolum: "Bilgisayar Muhendisligi" };
3
4 console.log(yeniKisi);
```

Listing 16: Nesnelerde spread kullanımı

2. Rest Operatörü (...)

* Birden fazla değeri tek değişkende toplamak için kullanılır.

```
1 function topla(...sayilar) {
2   let toplam = 0;
3
4   for (let sayi of sayilar) {
5     toplam += sayi;
6   }
7
8   return toplam;
9 }
10
11 console.log(topla(1, 2, 3, 4, 5));
```

Listing 17: Rest parametresi

7. Scope ve Hoisting

7.1. Scope

Scope, bir değişkenin erişilebilir olduğu alanı ifade eder. JavaScript'te genel olarak global scope, function scope ve block scope bulunur.

```
1 let globalDegisken = "Genel alan";
2
3 function test() {
4     let yerelDegisken = "Fonksiyon alanı";
5     console.log(globalDegisken);
6     console.log(yerelDegisken);
7 }
8
9 test();
```

Listing 18: Scope örneği

7.2. Block Scope

let ve const, blok içinde tanımlandığında yalnızca o blok içinde geçerlidir.

```
1 if (true) {
2     let mesaj = "Merhaba";
3     console.log(mesaj);
4 }
```

Listing 19: Block scope örneği

7.3. Hoisting

Hoisting, JavaScript'in değişken ve fonksiyon tanımlarını bellekte önceden hazırlaması ile ilgilidir. Ancak let ve const ile tanımlanan değişkenler kullanılmadan önce tanımlanmamıştır.

```
1 function selamla() {
2     console.log("Merhaba");
3 }
4
5 selamla();
```

Listing 20: Hoisting örneği

8. Callback Fonksiyonları

Bir fonksiyonun başka bir fonksiyona parametre olarak gönderilmesine callback yaklaşımı denir.

```
1 function islemYap(a, b, callback) {
2     return callback(a, b);
3 }
4
5 function topla(x, y) {
6     return x + y;
7 }
8
9 console.log(islemYap(3, 5, topla));
```

Listing 21: Callback örneği

Olay yönetiminde de callback yapısı sık kullanılır.

9. Hata Yönetimi

Program çalışırken ortaya çıkan hatalar kontrollü biçimde yakalanabilir.

9.1. try-catch

```
1 try {
2     let sonuc = 10 / 0;
3     console.log(sonuc);
4 } catch (hata) {
```

```
5 console.log("Bir hata olustu:", hata);
6 }
```

Listing 22: try-catch örneği

9.2. finally

```
1 try {
2     console.log("Islem basladi");
3 } catch (hata) {
4     console.log(hata);
5 } finally {
6     console.log("Islem tamamlandi");
7 }
```

Listing 23: finally örneği

9.3. throw

```
1 function yasKontrol(yas) {
2     if (yas < 0) {
3         throw new Error("Yas negatif olamaz");
4     }
5     return "Gecerli yas";
6 }
7
8 console.log(yasKontrol(20));
```

Listing 24: throw örneği

10. Tarih ve Zaman İşlemleri

JavaScript'te tarih ve saat bilgileri `Date` nesnesi ile yönetilir.

```
1 let simdi = new Date();
2
3 console.log(simdi);
4 console.log(simdi.getFullYear());
5 console.log(simdi.getMonth() + 1);
6 console.log(simdi.getDate());
7 console.log(simdi.getHours());
8 console.log(simdi.getMinutes());
```

11. DOM'da İleri İşlemler

Temel DOM işlemlerinden sonra daha esnek seçim ve düzenleme işlemleri yapılabilir.

11.1. `querySelector()` ve `querySelectorAll()`

```
1 let baslik = document.querySelector("h1");
2 console.log(baslik);
```

Listing 26: `querySelector` kullanımı

```
1 let paragraflar = document.querySelectorAll("p");
2
3 paragraflar.forEach(function(paragraf) {
4     console.log(paragraf.textContent);
5 });
```

Listing 27: `querySelectorAll` kullanımı

11.2. Class Ekleme ve Silme

```
1 let kutu = document.querySelector(".kutu");
2
3 kutu.classList.add("aktif");
4 kutu.classList.remove("aktif");
5 kutu.classList.toggle("goster");
```

Listing 28: `classList` kullanımı

11.3. Yeni Eleman Oluşturma

```
1 let yeniMadde = document.createElement("li");
2 yeniMadde.textContent = "Yeni gorev";
3
4 document.querySelector("ul").appendChild(yeniMadde);
```

Listing 29: Yeni eleman oluşturma

11.4. Eleman Silme

```
1 let madde = document.querySelector("li");
2 madde.remove();
```

Listing 30: Eleman silme

12. Form Doğrulama

Form verilerinin kontrol edilmesi web uygulamalarında temel işlemlerden biridir.

```
1 let form = document.querySelector("form");
2 let giris = document.querySelector("#ad");
3 let mesaj = document.querySelector("#mesaj");
4
5 form.addEventListener("submit", function(event) {
6     event.preventDefault();
7
8     if (giris.value.trim() === "") {
9         mesaj.textContent = "Ad alanı boş bırakılamaz.";
10    } else {
11        mesaj.textContent = "Form başarıyla gönderildi.";
12    }
13 });
```

Listing 31: Basit form doğrulama

13. localStorage ve sessionStorage

Tarayıcı üzerinde veri saklamak için `localStorage` ve `sessionStorage` kullanılır.

13.1. localStorage

```
1 localStorage.setItem("kullaniciAdi", "Ahmet");
2
3 let veri = localStorage.getItem("kullaniciAdi");
4 console.log(veri);
5
6 localStorage.removeItem("kullaniciAdi");
```

Listing 32: localStorage kullanımı

13.2. Nesne Saklama

Nesneler doğrudan saklanamaz. Önce JSON biçimine çevrilir.

```
1 let ogrenci = {
2     ad: "Ayse",
3     bolum: "Bilgisayar Muhendisligi"
4 };
5
6 localStorage.setItem("ogrenci", JSON.stringify(ogrenci));
7
8 let kayitliVeri = JSON.parse(localStorage.getItem("ogrenci"));
9 console.log(kayitliVeri);
```

Listing 33: JSON ile veri saklama

14. JSON Yapısı

JSON, veri taşımak ve saklamak için kullanılan metin tabanlı bir formattır.

```
1 let kisi = {
2     ad: "Mehmet",
3     yas: 25
4 };
5
6 let jsonVeri = JSON.stringify(kisi);
7 console.log(jsonVeri);
8
9 let tekrarNesne = JSON.parse(jsonVeri);
10 console.log(tekarNesne);
```

Listing 34: JSON dönüştürme işlemleri

15. Asenkron JavaScript

Bazı işlemler hemen tamamlanmaz. Özellikle internetten veri alma ve zamanlayıcı işlemleri asenkron yapı ile ilgilidir.

15.1. setTimeout

```
1 console.log("Baslangic");
2
```

```
3 setTimeout(function() {
4   console.log("2 saniye sonra calisti");
5 }, 2000);
6
7 console.log("Bitis");
```

Listing 35: setTimeout örneği

15.2. Promise

Promise, gelecekte tamamlanacak bir işlemin sonucunu temsil eder.

```
1 let kontrol = new Promise(function(resolve, reject) {
2   let basarili = true;
3
4   if (basarili) {
5     resolve("Islem basarili");
6   } else {
7     reject("Islem basarisiz");
8   }
9 });
10
11 kontrol
12   .then(function(sonuc) {
13     console.log(sonuc);
14   })
15   .catch(function(hata) {
16     console.log(hata);
17   });
```

Listing 36: Promise örneği

15.3. async ve await

```
1 function veriGetir() {
2   return new Promise(function(resolve) {
3     setTimeout(function() {
4       resolve("Veri alindi");
5     }, 2000);
6   });
7 }
8
9 async function calistir() {
10
11   let sonuc = await veriGetir();
12   console.log(sonuc);
13 }
14 calistir();
```

Listing 37: async-await örneği

16. API Kullanımı ve fetch

JavaScript ile internet üzerindeki servislerden veri alınabilir. Bu işlem için yaygın olarak `fetch()` kullanılır.

```
1 fetch("https://jsonplaceholder.typicode.com/users")
2   .then(function(response) {
3     return response.json();
4   })
5   .then(function(data) {
6     console.log(data);
7   })
8   .catch(function(error) {
9     console.log("Hata:", error);
10  });
```

Listing 38: fetch ile veri alma

17. Modüller

Büyük uygulamalarda kod tek bir dosyada tutulmaz. Kodlar farklı dosyalara ayrılır ve modüler yapı oluşturulur.

17.1. export

```
1 export function topla(a, b) {
2   return a + b;
3 }
```

Listing 39: export kullanımı

17.2. import

```
1 import { topla } from "../matematik.js";
2
3 console.log(topla(3, 5));
```

Listing 40: import kullanımı

18. Class Yapısı

JavaScript'te nesne yönelimli yaklaşım için `class` yapısı kullanılabilir.

```
1 class Ogrenci {
2     constructor(ad, yas) {
3         this.ad = ad;
4         this.yas = yas;
5     }
6
7     bilgiGoster() {
8         console.log(this.ad + " - " + this.yas);
9     }
10 }
11
12 let ogrenci1 = new Ogrenci("Ayse", 21);
13 ogrenci1.bilgiGoster();
```

Listing 41: class örneği

18.1. Kalıtım

```
1 class Kisi {
2     constructor(ad) {
3         this.ad = ad;
4     }
5
6     selamVer() {
7         console.log("Merhaba " + this.ad);
8     }
9 }
10
11 class Ogrenci extends Kisi {
12     constructor(ad, bolum) {
13         super(ad);
14         this.bolum = bolum;
```

```
15     }
16 }
17
18 let ogrenci = new Ogrenci("Zeynep", "Bilgisayar Muhendisligi");
19 ogrenci.selamVer();
```

Listing 42: Kalıtım örneği

19. Event Bubbling ve Event Delegation

DOM olaylarında bir olay önce hedef elemanda başlar, sonra üst elemanlara doğru yayılabilir. Bu davranışa event bubbling adı verilir.

```
1 document.querySelector(".dis").addEventListener("click", function
2     () {
3         console.log("Dis kutu tiklandi");
4     });
5
6 document.querySelector(".ic").addEventListener("click", function
7     () {
8         console.log("Ic kutu tiklandi");
9     });
```

Listing 43: Event bubbling örneği

Birden fazla benzer eleman için tek merkezden olay yönetimi yapıldığında event delegation yaklaşımı kullanılır.

```
1 document.querySelector("ul").addEventListener("click", function(
2     event) {
3     if (event.target.tagName === "LI") {
4         console.log(event.target.textContent);
5     }
6 });
```

Listing 44: Event delegation örneği

20. Modern Geliştirme Ortamı

JavaScript uygulamaları modern araçlarla birlikte geliştirilir. Paket yönetimi, modül yapısı ve derleme süreçleri bu aşamada önem kazanır.

20.1. npm

npm, JavaScript paketlerinin kurulmasını ve yönetilmesini sağlar.

```
1 npm init -y
```

Listing 45: npm başlangıç örneği

20.2. Paket Kurulumu

```
1 npm install axios
```

Listing 46: Paket yükleme örneği

20.3. Vite

Modern ön yüz uygulamalarında hızlı geliştirme ortamı sağlamak için Vite kullanılabilir.

```
1 npm create vite@latest
```

Listing 47: Vite proje oluşturma

21. TypeScript’e Giriş

TypeScript, JavaScript üzerine tip sistemi ekleyen bir yapıdır. Büyük projelerde veri tiplerinin açık biçimde belirtilmesi, hata yakalamayı kolaylaştırır.

```
1 let ad: string = "Ahmet";
2 let yas: number = 22;
3
4 function topla(a: number, b: number): number {
5     return a + b;
6 }
7
8 console.log(topla(3, 5));
```

Listing 48: TypeScript örneği